WanderMate: Social Mobile Travel Planner

Task 1

Requirements & Subsystem Overview

Contents

1 Functional Requirements **2**
- 1.1 Trip Itinerary Management . . . 2
- 1.2 POI Discovery & Place Details . . . 3
- 1.3 Route Visualisation . . . 3
- 1.4 Budget Tracking . . . 4
- 1.5 Social Feed & Community . . . 5
- 1.6 Collaborative Editing & Authentication . . . 6

2 Non-Functional Requirements **7**
- 2.1 Performance . . . 7
- 2.2 Scalability . . . 8
- 2.3 Availability & Reliability . . . 9
- 2.4 Security . . . 9
- 2.5 Usability & Maintainability . . . 10

3 Architecturally Significant Requirements — Summary **11**

4 Subsystem Overview **12**
- 4.1 Subsystem Summary Table . . . 13
- 4.2 Detailed Subsystem Descriptions . . . 14
  - 4.2.1 Mobile Client (React Native / Expo) . . . 14
  - 4.2.2 API Gateway (Node.js + Express) . . . 15
  - 4.2.3 POI & Routing Service (TravelService Facade) . . . 16
  - 4.2.4 Itinerary & Budget Engine . . . 16
  - 4.2.5 Social Feed & Community Service . . . 17
  - 4.2.6 Authentication & User Profile . . . 18
  - 4.2.7 Real-time Sync Infrastructure (Firebase — Cross-cutting) . . . 18

5 Subsystem Interaction Flows **19**

1. Functional Requirements

Functional requirements define what the WanderMate system must *do* — the behaviours and capabilities that directly serve user goals. The 23 requirements below are derived from the six core feature areas identified in the project proposal. Each is assigned a unique identifier, a category, a description, and a priority level (**High** / Medium).

High	Critical path; must be implemented in the prototype.
Medium	Important but deferrable to the final release.

1.1 Trip Itinerary Management

> **Architectural significance.** The itinerary is the central domain entity that every other subsystem reads from or writes to. Its schema design drives the MongoDB data model (nested `days[]` and `stops[]` subdocuments within the `Trip` document), the Firebase Realtime Database structure for collaborative sync, the AsyncStorage-backed offline cache, and the Zustand-based undo/redo stack.

Req ID	Category	Requirement	Priority
FR-01	Itinerary	Users shall be able to create a new trip with a name, destination, and date range. The system shall automatically generate a `days[]` array with one entry per calendar day in the range.	**High**
FR-02	Itinerary	Users shall be able to add, edit, remove, and reorder stops within any day of a trip. Reordering is supported via dedicated reorder API endpoints and move-up/move-down controls in the Explore screen's itinerary panel.	**High**
FR-03	Itinerary	The system shall persist itinerary data locally via AsyncStorage (using the `localCache` module) and queue mutations for server synchronisation when connectivity is restored (using the `syncQueue` module).	**High**
FR-04	Itinerary	Users shall be able to perform undo and redo on itinerary editing actions (add stop, remove stop, reorder, update trip). The `tripStore` maintains ordered `undoStack` and `redoStack` arrays to support multi-level undo/redo.	Medium

1.2 POI Discovery & Place Details

> **Architectural significance.** The dual-API design uses Google Places as the primary POI provider (for search, category browsing, ratings, photos, and opening hours) with the OpenRouteService (ORS) POI endpoint as a fallback when Google Places returns no results. An Overpass API adapter was initially developed but is currently disabled; the Facade pattern allows it to be re-enabled without changes to calling code. A server-side `node-cache` caching layer controls free-tier quota consumption.

Req ID	Category	Requirement	Priority
FR-05	POI Discovery	Users shall be able to search for hotels, restaurants, landmarks, and activities by keyword and category. The primary search is served by the Google Places Text Search and Nearby Search APIs; when Google Places returns zero results, the system falls back to the ORS POI endpoint.	**High**
FR-06	POI Discovery	The system shall display ratings, photos, opening hours, and price levels for each POI via the Google Places API. The `PlacesAdapter` normalises these fields into a common `POI` interface shared across all providers.	**High**
FR-07	POI Discovery	API responses from Google Places and ORS shall be cached server-side using `node-cache` (30-minute TTL for search results, 24-hour TTL for place details) to reduce latency and respect free-tier quotas.	**High**
FR-08	POI Discovery	Users shall be able to add a discovered POI to any day of their active itinerary with a single tap via the "Add to Trip" button on each POI card in the Explore screen.	**High**

1.3 Route Visualisation

> **Architectural significance.** Embedding Leaflet inside a React Native `WebView` creates a cross-boundary JavaScript bridge. Route optimisation calls OpenRouteService (ORS) and must respect its daily quota of 2 000 requests. The ORS adapter implements both simple route calculation and multi-stop optimisation via the ORS `/optimization` endpoint (VROOM solver), with caching and rate limiting at the API Gateway layer.

Req ID	Category	Requirement	Priority
FR-09	Routing	The system shall render an interactive map of each day's itinerary using Leaflet and OpenStreetMap tiles inside a React Native `WebView`. Both the dedicated map screen (`trip/map/[id].tsx`) and the integrated Explore screen map mode use this approach.	**High**
FR-10	Routing	The system shall compute an optimised multi-stop route for each itinerary day via the ORS Directions API (`/v2/directions/{profile}/geojson`) and overlay the GeoJSON polyline on the map. For more than two stops, the ORS Optimization API (`/optimization`) determines the optimal visit order before computing the route.	**High**
FR-11	Routing	The map shall update automatically as stops are added, removed, or reordered in the Explore screen's itinerary panel. The Leaflet WebView re-renders whenever the underlying trip state (managed by `tripStore`) changes, without requiring a manual refresh.	Medium

1.4 Budget Tracking

> **Architectural significance.** Expense data is stored in a dedicated `Expense` MongoDB collection, referenced by `trip` ObjectId. The budget API computes per-day summaries, category breakdowns, and per-person split amounts server-side (deterministic logic). Real-time synchronisation across collaborators is achieved via Firebase Realtime Database listeners on `budgets/{tripId}/lastUpdate`.

Req ID	Category	Requirement	Priority
FR-12	Budget	Users shall be able to log individual expenses, each tagged with a category (accommodation, food, transport, activities, other) and associated with a specific trip. The Explore screen also supports inline cost entry per stop, which automatically creates a corresponding expense.	**High**

Req ID	Category	Requirement	Priority
FR-13	Budget	The system shall display a total trip budget, per-category breakdowns with progress bars, and a per-person share. The budget summary is computed server-side in the `GET /api/budget/:tripId` endpoint and rendered in the Budget tab with visual category cards.	**High**
FR-14	Budget	Users shall be able to split a group trip's total expenses equally among all participants (owner + collaborators). The per-person share is calculated server-side and displayed in the Budget summary card.	Medium
FR-15	Budget	Budget data shall synchronise across all collaborators on a shared trip in real time. The `budgetStore` subscribes to `budgets/{tripId}/lastUpdate` via a Firebase `onValue` listener; any update triggers a re-fetch of the full budget from the API.	Medium

1.5 Social Feed & Community

> **Architectural significance.** The social feed is backed by a dedicated `FeedPost` MongoDB collection with indexes on `createdAt` and `likeCount` for efficient pagination and sorting. Feed posts are denormalised snapshots (trip name, destination, budget summary, stop count) rather than live references, ensuring feed rendering is independent of trip mutations. Cursor-based pagination is implemented via `skip/limit` queries.

Req ID	Category	Requirement	Priority
FR-16	Social Feed	Users shall be able to publish a completed trip to a public social feed. Each feed post displays the author name and avatar, trip name, destination, duration, participant count, total budget, and stop count. The `POST /api/feed/publish/:tripId` endpoint atomically creates the `FeedPost` document, marks the trip as published, and writes a Firebase event for real-time propagation.	**High**

Req ID	Category	Requirement	Priority
FR-17	Social Feed	Users shall be able to follow and unfollow other travellers, and browse a personalised "Following" feed filtered to posts by followed users. The feed screen provides a `SegmentedButtons` toggle between "Discover" (all posts) and "Following" modes.	**High**
FR-18	Social Feed	Users shall be able to like and unlike published itineraries. Like counts are visible on feed cards and toggle state is persisted per-user via a `likes[]` array on the `FeedPost` document.	Medium
FR-19	Social Feed	Users shall be able to clone any public itinerary into their own trip list as an independently editable copy. The `POST /api/feed/:postId/clone` endpoint deep-copies the original trip's `days[]` and `stops[]` subdocuments, assigns the cloning user as the new owner, clears the `isPublished` flag, and writes the new trip atomically.	**High**

1.6 Collaborative Editing & Authentication

Architectural significance. Real-time collaborative editing uses Firebase Realtime Database as an event bus. When any collaborator mutates a trip via the API Gateway, the backend writes an update event to `trips/{tripId}` in Firebase. All connected clients subscribe via `onValue` listeners (Observer pattern) and re-fetch the authoritative trip state from MongoDB. Authentication supports both Firebase Email/Password and Google Sign-In (OAuth 2.0); session persistence across app restarts is achieved via `getReactNativePersistence(AsyncStorage)`.

Req ID	Category	Requirement	Priority
FR-20	Collaboration	Users shall be able to invite other registered users to co-edit a trip. The `POST /api/trips/:id/collaborators` endpoint adds a user's Firebase UID to the trip's `collaborators[]` array and broadcasts the update to Firebase. Changes by any collaborator propagate to all others in real time via Firebase `onValue` listeners in `tripStore.subscribeTripUpdates()`.	**High**

Req ID	Category	Requirement	Priority
FR-21	Collaboratio	The system shall resolve concurrent edits using last-write-wins semantics. All mutations via the API Gateway are timestamped server-side (`updatedAt: Date.now()`) in both MongoDB and Firebase to establish a total order.	**High**
FR-22	Auth	Users shall be able to register and sign in using either email/password credentials or Google Sign-In via Firebase Authentication (OAuth 2.0). The sign-in screen provides both options with a toggle between "Sign In" and "Create Account" modes.	**High**
FR-23	Auth	Authenticated sessions shall persist across app restarts and device reboots. On native platforms, Firebase Auth is initialised with `getReactNativePersistence(AsyncStorage)` ensuring the user remains signed in until explicit sign-out.	Medium

2. Non-Functional Requirements

Non-functional requirements (NFRs) govern the quality attributes of WanderMate beyond feature correctness. Each NFR below is architecturally significant — it has a direct impact on technology selection, pattern adoption, or structural decomposition, and each is accompanied by a concrete, measurable target.

2.1 Performance

> **Architectural impact.** NFR-01 and NFR-02 drive the `node-cache` caching layer inside the TravelService Facade: 30-minute TTL for search results and 1-hour TTL for route data. NFR-03 justifies the event-driven Firebase approach over a REST polling model, which would introduce noticeable lag for collaborators and unnecessary bandwidth consumption.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-01	Performance	The app shall render the itinerary map and return POI search results within 2 seconds on a 4G mobile connection.	< 2 s at p95

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-02	Performance	The social feed shall paginate results (10 posts per page) and display the first page of cards within 1.5 seconds of the Feed tab being opened.	< 1.5 s first-page load
NFR-03	Performance	Real-time itinerary and budget updates shall propagate to all active collaborators within 500 ms of a change being committed to Firebase.	< 500 ms sync latency

2.2 Scalability

> **Architectural impact.** NFR-06 is the primary economic constraint of the entire architecture. It is the central driver of the TravelService Facade (to centralise quota control), the `node-cache` caching strategy (to minimise outbound API calls), and the `express-rate-limit` middleware applied to all external API proxy routes.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-04	Scalability	The Node.js + Express API Gateway shall support at least 200 concurrent users without degradation in response time beyond the targets set in NFR-01 and NFR-02.	200 concurrent users
NFR-05	Scalability	The MongoDB data schema shall support horizontal sharding by user ID to accommodate growth beyond 10 000 registered users without schema migration. Indexes on `owner`, `collaborators`, `firebaseUid`, and `createdAt` are defined to support this.	10 000+ users
NFR-06	Scalability	The system shall operate within free-tier API quotas at launch: Google Places ($\leq$ 5 000 req/month), ORS ($\leq$ 2 000 req/day), achieved through server-side `node-cache` caching and per-user `express-rate-limit` rate limiting (100 general req/15 min, 50 external API req/hour).	Zero overage cost at launch

2.3 Availability & Reliability

> **Architectural impact.** NFR-07 mandates the AsyncStorage-backed `localCache` and `syncQueue` modules on the client. NFR-08 is implemented via the Strategy pattern in `TravelService`: when Google Places fails, the search method falls back to ORS POI results without any change to the calling code or UI layer.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-07	Availability	Core itinerary viewing and editing shall remain available when the device is offline. The `tripStore` reads from `localCache` when API calls fail, and mutations are queued in `syncQueue` for server sync on reconnect, with no data loss.	100% offline read/write for cached data
NFR-08	Reliability	If the Google Places API is unavailable, POI search shall degrade gracefully to ORS POI results; no user-facing crash or blank screen shall occur. The `TravelService.searchPOI()` method wraps each provider call in a try/catch and silently falls through to the next provider.	Zero crashes on external API failure
NFR-09	Reliability	The API Gateway shall return a structured error response (HTTP 4xx/5xx with a JSON `{error}` body) for every failure scenario. A global Express error handler and per-route try/catch blocks ensure no unhandled exceptions propagate to clients.	100% structured error responses

2.4 Security

> **Architectural impact.** NFR-11 is the primary reason the API Gateway subsystem exists as a standalone service rather than being embedded in the mobile client. The gateway acts as a secret vault, keeping all third-party API keys (Google Places, ORS) server-side as environment variables and never exposing them in the mobile app bundle. Firebase Auth JWT verification is performed by custom middleware on every authenticated route.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-10	Security	All API communication between the mobile client and the backend shall use HTTPS; plaintext HTTP connections shall be rejected.	TLS 1.2+ enforced on all channels
NFR-11	Security	Third-party API keys (Google Places, OpenRouteService) shall never be embedded in the mobile app bundle. All calls to external APIs shall be proxied through the backend API Gateway, which stores keys in `.env` environment variables.	Zero secrets in client bundle
NFR-12	Security	Backend authorisation middleware enforces per-trip access control: only the trip `owner` and users listed in the `collaborators[]` array can read or write trip data. Owner-only actions (delete trip, add collaborators, publish) are additionally enforced.	Role-based access enforced at API layer

2.5 Usability & Maintainability

> **Architectural impact.** NFR-15 codifies the Facade + Strategy design decision as a non-negotiable maintainability requirement. It ensures that future provider changes (e.g., re-enabling Overpass, replacing ORS) do not propagate breaking changes through the React Native UI layer, protecting the product from vendor lock-in.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-13	Usability	Any primary feature of the app (trips list, explore/map, budget tracker, social feed, profile) shall be reachable from any screen in no more than 3 taps via the bottom tab bar.	Max 3 taps to any primary feature

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-14	Maintainability	The codebase shall be organised into independently testable modules aligned with subsystem boundaries. The frontend uses feature-based stores (`tripStore`, `budgetStore`, `feedStore`, `authStore`) and the backend uses feature-based route/model modules (trips, budget, feed, users, POI, routes).	Module fan-out < 5 per module
NFR-15	Maintainability	Swapping the POI provider (e.g., re-enabling Overpass, replacing ORS with an alternative) shall require code changes only within the `TravelService` Facade and the relevant adapter module; no changes to UI screens, API Gateway routes, or stores shall be necessary.	Facade fully isolates provider logic

3. Architecturally Significant Requirements — Summary

The following requirements are designated **Architecturally Significant Requirements (ASRs)** because they impose cross-cutting constraints, force the adoption of specific patterns or infrastructure components, or have an outsized influence on overall system structure. Failure to address any ASR would compromise the feasibility, performance, or maintainability of the entire system.

Req ID	Driver	Architectural Impact
FR-03	Offline Persistence & Sync	Mandates the `localCache` and `syncQueue` modules backed by AsyncStorage. Shapes the client-side data flow: stores attempt API calls first, falling back to cache on failure and queuing mutations for later sync.
FR-07	API Response Caching	Drives the `node-cache`-based caching layer inside the TravelService Facade. Directly mitigates the risk of exceeding free-tier API quotas (Google Places, ORS).

Req ID	Driver	Architectural Impact
FR-19	Itinerary Clone	Requires the Social Feed and Itinerary subsystems to share a common trip schema so that cloned documents are immediately editable without data transformation. Implemented as a deep-copy operation in the `clone` route.
FR-20 / FR-21	Real-time Collaboration	Justifies Firebase Realtime Database over a REST polling approach. Requires the Observer pattern with Firebase `onValue` listeners in `tripStore` and `budgetStore`.
NFR-06	Free-tier Quota Compliance	The central economic constraint of the architecture. Drives both the Facade (central quota control), the `node-cache` caching (minimise outbound calls), and the `express-rate-limit` middleware (enforce per-user limits).
NFR-07	Offline-first Availability	Elevates offline capability from a post-launch enhancement to a first-class architectural concern, requiring the `syncQueue` and `localCache` services with conflict-resolution via last-write-wins.
NFR-11	No Client-side Secrets	Mandates the API Gateway as a standalone subsystem. Without it, third-party API keys (Google Places, ORS) would be exposed in the mobile app bundle.
NFR-15	Provider Swappability	Enforces the Facade + Strategy pattern as a hard maintainability requirement. The `TravelService` class wraps all provider adapters, preventing UI components and API routes from taking a direct dependency on any specific POI provider.

4. Subsystem Overview

WanderMate is decomposed into **six primary subsystems** and one cross-cutting infrastructure layer. Each subsystem is independently deployable and testable, communicates through well-defined interfaces, and maps directly to one or more functional requirement groups defined in Section 1.

The decomposition follows three guiding principles:

- **Single Responsibility** — each subsystem owns one cohesive slice of domain logic.
- **Low Coupling / High Cohesion** — inter-subsystem communication goes through explicit interface contracts (REST endpoints, Firebase paths, defined service APIs, Zustand store boundaries).
- **Replaceability** — the POI provider, the mapping library, and the real-time infrastructure can each be replaced without restructuring other subsystems.

4.1 Subsystem Summary Table

Subsystem	Role & Responsibility	Key Components	Interfaces With
Mobile Client (React Native / Expo)	Entry point for all user interactions. Renders the five bottom-tab screens (Trips, Explore, Feed, Budget, Profile), manages local state via Zustand stores, hosts the Leaflet WebView map, and orchestrates calls to the API Gateway and Firebase.	Expo Router (tabs + stack), Leaflet WebView, Zustand stores (`tripStore`, `budgetStore`, `feedStore`, `authStore`), `localCache`, `syncQueue`, Firebase SDK client, React Native Paper UI	API Gateway, Firebase Realtime DB, Firebase Auth
API Gateway (Node.js + Express)	Single backend entry point. Proxies and authenticates all third-party API calls, enforces rate limits, applies caching, and serves trip/user/feed/budget data from MongoDB. Keeps all secrets server-side.	Express routes (`trips`, `users`, `feed`, `budget`, `poi`, `routes`), Firebase JWT middleware, `express-rate-limit`, `node-cache`, Mongoose models, Helmet security headers	Mobile Client, MongoDB, Google Places API, ORS
POI & Routing Service (TravelService Facade)	Encapsulates the dual-API POI strategy (Google Places as primary, ORS POI as fallback) and ORS route optimisation behind a single `TravelService` interface. Implements response caching and provider fallback. An Overpass adapter exists but is currently disabled.	`TravelService` class (singleton), `PlacesAdapter`, `ORSAdapter`, `overpassAdapter` (disabled), `node-cache` instance	API Gateway, Google Places API, ORS

Subsystem	Role & Responsibility	Key Components	Interfaces With
Itinerary & Budget Engine	Manages the full lifecycle of trip documents: CRUD, stop management (add, edit, remove, reorder), expense logging, split calculation, and offline sync queue. Undo/redo is implemented via command stacks in `tripStore`.	`Trip` model, `Expense` model, trip routes, budget routes, `tripStore` (undo/redo), `budgetStore`, `syncQueue`, Firebase write listeners	API Gateway, Firebase Realtime DB, AsyncStorage
Social Feed & Community Service	Handles trip publishing, follower graph management, like/unlike toggle, feed pagination, and one-tap itinerary cloning. Publishes update events to Firebase for real-time card refresh.	`FeedPost` model, feed routes, `feedStore`, `User.followers[]`/`following[]` arrays, user routes (follow/unfollow/search)	API Gateway, Firebase Realtime DB, Itinerary Engine
Auth & User Profile	Manages user identity via Firebase Authentication (Email/Password + Google Sign-In), stores user profile data in MongoDB, and enforces per-trip access control via backend authorisation middleware.	Firebase Auth SDK (`firebase/auth`), `expo-auth-session` (Google OAuth), `User` model, `authStore` (Zustand), `authenticate` middleware (Firebase Admin JWT verification)	Firebase Auth, API Gateway, MongoDB
Real-time Sync Infrastructure (Firebase — cross-cutting)	Cross-cutting layer providing event-driven data propagation for collaborative itinerary edits, budget updates, and social feed events. All reactive subsystems register Firebase `onValue` listeners rather than polling.	Firebase Realtime Database, Firebase Admin SDK (backend write), Firebase client SDK (frontend read), connection-state listeners	Itinerary Engine, Budget Engine, Social Feed Service, Mobile Client

4.2 Detailed Subsystem Descriptions

4.2.1 Mobile Client (React Native / Expo)

The mobile client is the sole user-facing component of WanderMate. It is built with React Native using the Expo managed workflow (SDK 54), targeting Android and web.

Navigation is provided by Expo Router with a bottom tab bar containing five primary screens: **My Trips** (trips list with create/set-active actions), **Explore** (POI search + integrated map + itinerary panel via `@gorhom/bottom-sheet`), **Feed** (social feed with discover/following toggle), **Budget** (expense tracking with category breakdowns), and **Profile** (user stats, offline sync trigger, sign-out).

The interactive map is rendered by embedding Leaflet 1.9.4 inside a `WebView` component. The map uses OpenStreetMap tiles and displays numbered markers with category-coloured borders. A bidirectional JavaScript bridge passes itinerary data into the WebView and receives user interactions (pin clicks, stop selection) out via `postMessage`/`onMessage`. Route polylines from ORS are rendered as GeoJSON overlays; when ORS route data is unavailable, a dashed straight-line fallback is drawn.

State management uses **Zustand** with five dedicated stores: `authStore` (Firebase auth state, profile), `tripStore` (trip CRUD, stop management, undo/redo stacks, Firebase subscription), `budgetStore` (expense CRUD, summary, Firebase subscription), `feedStore` (feed pagination, like/clone/publish), and `activeTripStore` (tracks which trip is "active" for the Explore screen).

The `localCache` module provides an offline-first data layer using AsyncStorage: when the device is offline, stores read from cache and write mutations to the `syncQueue` for server sync on reconnect. All outbound API requests carry a Firebase JWT obtained at login via the Axios request interceptor in `services/api.ts`.

> **Key patterns:** Observer (Firebase `onValue` listeners for live trip and budget updates), Command (undo/redo via `undoStack`/`redoStack` in `tripStore`), Cache-Aside (`localCache` abstraction over AsyncStorage).
> **Satisfies:** FR-01 to FR-04, FR-08 to FR-11, NFR-07, NFR-13.

4.2.2 API Gateway (Node.js + Express)

The API Gateway is the single backend service in WanderMate's architecture. Its primary architectural function is to act as a *secret vault* — all third-party API keys (Google Places, ORS) reside here as environment variables in `.env` and are never transmitted to or stored on the mobile client.

In addition to secret management, the gateway performs: Firebase JWT verification via custom `authenticate` middleware (using Firebase Admin SDK's `verifyIdToken`), per-user rate limiting via `express-rate-limit` (100 general requests per 15 minutes, 50 external API requests per hour), a `node-cache` in-memory cache with configurable TTLs to reduce duplicate outbound calls, Helmet security headers on all responses, and CRUD REST endpoints for six resource types backed by MongoDB via Mongoose ODM:

- `/api/trips` — trip CRUD, stop management, collaborator management
- `/api/budget` — expense CRUD, budget summaries with split calculation
- `/api/feed` — feed pagination, publish, like/unlike, clone
- `/api/users` — profile CRUD, follow/unfollow, user search
- `/api/poi` — POI search and category browsing (proxied via TravelService)
- `/api/routes` — route calculation and optimisation (proxied via TravelService)

A global error handler ensures structured JSON error responses for all failure scenarios (NFR-09), and a 404 handler catches undefined endpoints.

> **Key patterns:** Facade (single entry point abstracting all third-party APIs), Singleton (TravelService instance).
> **Satisfies:** NFR-04, NFR-09, NFR-10, NFR-11.

4.2.3 POI & Routing Service (TravelService Facade)

This module implements the **Facade** and **Strategy** patterns. `TravelService` is instantiated as a singleton and exposes four methods to the gateway layer: `searchPOI(query, lat, lng, radius)`, `searchByCategory(category, lat, lng, radius)`, `getPlaceDetails(place name, lat, lng)`, and `getRoute(coordinates, profile)`. Internally, two active adapters implement provider-specific API logic:

- **PlacesAdapter** — primary POI provider. Queries Google Places Text Search, Nearby Search, Find Place, and Place Details APIs. Returns normalised POI objects with ratings, photos, opening hours, price levels, and addresses. Quota-sensitive: 5 000 req/month on free tier.
- **ORSAdapter** — dual-purpose adapter serving both as a POI fallback (via the ORS `/pois` endpoint, limited to 2 000m radius) and as the routing engine (via `/v2/directions` for route calculation and `/optimization` for multi-stop visit-order optimisation using the VROOM solver).
- **OverpassAdapter** — an OpenStreetMap data provider querying the Overpass API. The adapter is *implemented but currently disabled* (entire file commented out). It can be re-enabled by uncommenting and importing it in `TravelService`, satisfying NFR-15.

A shared `node-cache` instance sits in front of all adapter methods. Cache TTLs are: 30 minutes for search results, 24 hours for place details, and 1 hour for route calculations. If Google Places is unavailable at runtime, `TravelService` falls back to returning ORS POI results, satisfying NFR-08 without any change to the calling code.

> **Key patterns:** Facade (`TravelService` wraps multiple APIs behind a single interface), Strategy (swappable adapter implementations with a common normalised output format), Singleton (single `TravelService` instance shared across all routes).
> **Satisfies:** FR-05 to FR-08, FR-09 to FR-11, NFR-01, NFR-06, NFR-08, NFR-15.

4.2.4 Itinerary & Budget Engine

This subsystem owns the core domain logic of WanderMate. On the backend, the `Trip` Mongoose model defines a nested document structure: trips contain a `days[]` array, each day containing a `stops[]` array with fields for name, coordinates, category, order, notes, arrival time, duration, cost, and optional links to expense records. The `Expense` model stores expense records in a separate collection, referenced by trip ObjectId.

Backend routes implement full trip lifecycle management: create (with auto-generated days array), read (with owner/collaborator authorisation), update, delete (owner-only), add stop, update stop, remove stop, reorder stops, and add collaborator. Every mutation writes an update event to Firebase Realtime Database at `trips/{tripId}` for real-time sync.

On the frontend, `tripStore` (Zustand) manages all trip state and implements the **Command** pattern for undo/redo. Every mutation (add stop, remove stop, reorder, update trip) pushes a command object onto the `undoStack` containing the mutation

type, associated data, and the previous trip state. Undo restores the previous state via an API call; redo re-applies the undone operation. The `redoStack` is cleared whenever a new mutation is performed.

The `budgetStore` manages expense state and subscribes to Firebase `budgets/{tripId}/lastUpdate` for real-time budget sync across collaborators. Budget summaries (total, per-category, per-day, per-person split) are computed server-side in the `GET /api/budget/:tripId` endpoint.

A `syncQueue` persists unsent mutations to AsyncStorage when the device is offline and flushes them in timestamp order on reconnect. The Profile screen provides a manual "Sync Offline Changes" action.

> **Key patterns:** Command (undo/redo stacks in `tripStore`), Observer (Firebase `onValue` listeners for both trip and budget updates), Cache-Aside (`localCache` for offline reads).
>
> **Satisfies:** FR-01 to FR-04, FR-12 to FR-15, FR-20 to FR-21, NFR-03, NFR-07.

4.2.5 Social Feed & Community Service

The social feed subsystem manages the public layer of WanderMate. When a user publishes a trip, the `POST /api/feed/publish/:tripId` endpoint creates a `FeedPost` document containing denormalised metadata: author name and avatar, trip name, destination, cover image, date range, duration, participant count, total budget (summed from expenses), and stop count. This denormalised snapshot approach ensures feed card rendering is independent of trip mutations. The trip is marked as published (`isPublished: true`) and added to the user's `publishedTrips[]` array. A Firebase event is written to `feed/{postId}` for real-time propagation.

The feed endpoint supports pagination via `skip/limit` with two modes: "Discover" (all posts sorted by `createdAt`) and "Following" (posts by followed users only). The `feedStore` on the client manages pagination state and implements infinite scroll via `onEndReached`.

Like/unlike is implemented as a toggle: the `POST /api/feed/:postId/like` endpoint adds or removes the user's UID from the `likes[]` array and updates the denormalised `likeCount`.

The `POST /api/feed/:postId/clone` endpoint performs a deep copy of the original trip document: it copies all `days[]` and their `stops[]` subdocuments, assigns the cloning user as the new owner, clears the `isPublished` flag, resets collaborators, and writes the new trip atomically. This prevents partial clones and ensures the cloned trip is immediately editable (satisfies FR-19).

The follower graph is stored as adjacency lists in the `User` model: `followers[]` and `following[]` arrays of Firebase UIDs. Follow, unfollow, and user search endpoints are provided in the users route module.

> **Key patterns:** Observer (Firebase feed event broadcasting), Snapshot (denormalised feed post data for independent rendering).
>
> **Satisfies:** FR-16 to FR-19, NFR-02, NFR-03.

4.2.6 Authentication & User Profile

Identity management is handled by Firebase Authentication, supporting two providers: **Email/Password** (registration with display name, sign-in) and **Google Sign-In** (OAuth 2.0 via `expo-auth-session` on native, `signInWithPopup` on web). On native platforms, auth state persistence across app restarts is achieved via `getReactNativePersistence(Asy` (satisfies FR-23).

On first successful sign-in (via either method), the API Gateway's `GET /api/users/me` endpoint creates a corresponding `User` document in MongoDB containing the user's Firebase UID, email, display name, avatar URL, follower/following ID lists, and references to published trips. The `authStore` initialises a Firebase `onAuthStateChanged` listener on app startup to track the authentication state and automatically fetch the user profile.

Backend authorisation is enforced by the `authenticate` middleware, which verifies Firebase ID tokens on every API request using the Firebase Admin SDK's `verifyIdToken` method. Per-trip access control is additionally enforced in each route handler by checking the requesting user's UID against the trip's `owner` and `collaborators[]` fields.

> **Key patterns:** Observer (`onAuthStateChanged` listener in `authStore`), Singleton (Firebase app initialisation).
>
> **Satisfies:** FR-22, FR-23, NFR-10, NFR-11, NFR-12.

4.2.7 Real-time Sync Infrastructure (Firebase — Cross-cutting)

Firebase Realtime Database serves as the event bus for WanderMate's three real-time channels:

- **Collaborative itinerary editing** — every trip mutation via the API Gateway writes an update event to `trips/{tripId}` in Firebase (containing `updatedAt` and `lastEditedBy`). All connected clients register `onValue` listeners via `tripStore.subscribeTripUpd` which triggers a re-fetch of the authoritative trip state from MongoDB.
- **Budget synchronisation** — expense mutations write to `budgets/{tripId}/lastUpdate`. The `budgetStore.subscribeBudgetUpdates()` listener triggers a budget re-fetch on any change.
- **Social feed updates** — new published posts are written to a `feed/{postId}` path for real-time propagation.

This layer is intentionally kept thin. It carries events and deltas only; full document state remains authoritative in MongoDB. This architectural decision prevents Firebase from becoming a second source of truth for complex relational queries (follower graphs, expense aggregations), which it is not optimised for. Firebase's built-in offline queue and automatic reconnection complement the client-side `syncQueue` to provide robust offline-to-online transitions.

On the backend, Firebase is accessed via the Firebase Admin SDK (initialised as a singleton in `config/firebase.js`). On the frontend, the Firebase client SDK is initialised in `config/firebase.ts` with AsyncStorage-backed auth persistence.

> **Key patterns:** Observer (all reactive subsystems attach Firebase `onValue` listeners), Event Bus (Firebase paths act as named event channels).
>
> **Satisfies:** FR-15, FR-20, FR-21, NFR-03, NFR-07.

5. Subsystem Interaction Flows

The following five runtime data flows describe how subsystems collaborate to deliver WanderMate's core behaviours. These flows directly trace back to the ASRs identified in Section 3.

#	Flow	Step-by-step Description
1	POI & Route Query	Mobile Client (Explore screen) → API Gateway `/api/poi/search` (HTTPS + JWT) → TravelService Facade checks `node-cache`; on miss, calls PlacesAdapter (Google Places Text Search); on empty results, falls back to ORSAdapter POI search → cached and returned to client → user taps "Add to Trip" → `tripStore.addStop()` calls `/api/trips/:id/days/:dayIndex/stops` → route auto-computed via `/api/routes/optimize` (ORS Optimization API) → Leaflet WebView renders route polyline and numbered stop markers.
2	Collaborative Edit	Collaborator A edits a stop on Mobile Client → `tripStore` calls API Gateway, which updates the Trip document in MongoDB and writes `{updatedAt, lastEditedBy}` to Firebase `trips/{tripId}` → Firebase broadcasts to all registered `onValue` listeners → Collaborator B's `tripStore` receives the event and refetches the trip from MongoDB via the API → UI re-renders with updated data.
3	Feed Publish	User taps "Publish" in Trip Detail screen → `feedStore.publishTrip()` calls `POST /api/feed/publish/:tripId` → backend creates `FeedPost` document with denormalised metadata, marks trip as published, adds to user's `publishedTrips[]`, writes event to Firebase `feed/{postId}` → feed cards appear for all users on next feed fetch; followers receive real-time notification through Firebase.
4	Itinerary Clone	User taps "Clone Trip" on a feed card → `feedStore.cloneTrip()` calls `POST /api/feed/:postId/clone` → backend deep-copies the original trip's `days[]` and `stops[]`, assigns new owner, clears `isPublished` flag, saves new Trip document atomically, syncs to Firebase → new trip appears in user's Trips tab immediately.

#	Flow	Step-by-step Description
5	Offline Sync	Device goes offline → `tripStore` mutations fail API call, fall back to `localCache` for reads and queue mutations in `syncQueue` (AsyncStorage) → device reconnects → user triggers "Sync Offline Changes" from Profile screen (or `syncQueue.flush()` is called) → queued mutations are replayed in timestamp order to the API Gateway → MongoDB updated → Firebase receives resolved state → remote collaborators' views converge.